Osmania University — CSE/AIML

Machine Learning

Mid-Term Exam 1 & 2 — Complete Answers

Class Test-I | Class Test-II | B.E V-Semester

Part-A (Short Answers) + Part-B (Long Answers with Solved Problems)

Section	Topics
Mid-Term Test 1 — Part A	Supervised/Unsupervised/RL | Version Space | Bayes Rule | Decision Tree | FIND-S
Mid-Term Test 1 — Part B	Learning Example Design | Inductive Hypothesis | Candidate Elimination | MLE | MAP | Entropy & IG
Mid-Term Test 2 — Part A	Perceptron | Coin Problem (Bayes) | Regularization | LDA Steps | Reinforcement Learning
Mid-Term Test 2 — Part B	SVM & Kernels | Linear Regression (Solved) | Backpropagation | K-Means | MLP | LDA Projection (Solved)

Class Test-I — 2026 | Machine Learning

B.E V-Semester | Department of Computer Science and Engineering | Osmania University

Max Marks: 30 | Part-A: 5×2=10 | Part-B: 2×10=20

PART-A (5 × 2 = 10 Marks) — Short Answer Questions

Answer

Q1. Briefly explain any TWO: (i) Supervised Learning (ii) Unsupervised Learning (iii) Reinforcement Learning [2 marks]

(i) Supervised Learning:

In Supervised Learning, the algorithm learns from a labeled training dataset — each input example has a corresponding correct output (label). The model learns a mapping function f: X → Y such that it can predict Y for new unseen X.

> Examples: Classification: Email → Spam or Not-Spam (discrete labels) Regression: House features → Price (continuous output) Key algorithms: Decision Tree, SVM, Neural Networks, Logistic Regression Training: Model adjusts weights by comparing prediction vs actual label Goal: Minimize prediction error on unseen test data (generalization)

(ii) Unsupervised Learning:

In Unsupervised Learning, the algorithm learns patterns from unlabeled data — there are no predefined output labels. The model finds hidden structure or groupings by itself.

> Examples: Clustering: K-Means groups customers by purchasing behavior Dimensionality Reduction: PCA reduces 100 features to 10 Association: Market basket analysis — {bread, butter, jam} often bought together Key algorithms: K-Means, DBSCAN, PCA, Autoencoders

(iii) Reinforcement Learning:

An agent learns to make sequential decisions by interacting with an environment. It receives reward (positive) or penalty (negative) signals and learns to maximize cumulative reward over time through trial and error.

> Examples: Game playing: AlphaGo defeated world champion at Go Robotics: Robot learns to walk by trying movements Self-driving: Car learns to navigate through rewards/penalties Key terms: Agent, Environment, State, Action, Reward, Policy

---

Answer

Q2. Define with suitable example: "Version Space Representation Algorithm" [2 marks]

Version Space is the set of all hypotheses in hypothesis space H that are consistent with all training examples (correctly classifying all positive examples and no negative examples).

The **Candidate Elimination Algorithm** maintains two boundaries of the version space:

- **G — General boundary:** Most general hypotheses consistent with all negatives. Init: G = []
- **S — Specific boundary:** Most specific hypotheses consistent with all positives. Init: S = [<ø,ø,ø,ø,ø,ø>]

```
Version Space VS(H,D) = { h ∈ H : h is consistent with all examples in D }
Consistent: h(x)=c(x) for all training examples (x, c(x)) in D
```

Example — EnjoySport task (6 attributes: Sky,Temp,Humidity,Wind,Water,Forecast): Example 1 (+): -> YES S: [most specific] G: [still most general] Example 2 (+): -> YES S generalizes: Example 3 (-): -> NO G specializes to exclude this example Final Version Space is bounded between S and G boundaries.

Answer

Q3. Define Bayes Rule [2 marks]

Bayes Rule (Bayes Theorem) is a fundamental theorem in probability that describes how to update the probability of a hypothesis after observing evidence (data).

```
Bayes' Theorem: P(h | D) = [ P(D | h) * P(h) ] / P(D) P(h|D) = Posterior — probability of hypothesis h AFTER seeing data D P(D|h) = Likelihood — probability of observing data D IF h were true P(h) = Prior — initial probability of h BEFORE seeing data P(D) = Evidence — total probability of D across all hypotheses
```

MAP (Maximum a Posteriori) Hypothesis:

```
h_MAP = argmax_h P(D|h) * P(h) [most probable hypothesis given data]
```

Medical Example: P(Cancer) = 0.01 (prior — 1% prevalence) P(Test+ | Cancer) = 0.90 (test sensitivity) P(Test+ | No Cancer) = 0.10 (false positive rate) P(Test+) = 0.90*0.01 + 0.10*0.99 = 0.009 + 0.099 = 0.108 P(Cancer | Test+) = P(Test+|Cancer)*P(Cancer) / P(Test+) = 0.90*0.01 / 0.108 = 0.009/0.108 = 0.083 Interpretation: Even with positive test, only 8.3% chance of cancer! (Prior probability of 1% is very important)

Answer

Q4. Give Decision Tree to represent: (i) A∧B (ii) A∨[B∧C] [2 marks]

Decision Tree represents a Boolean function by a tree structure where internal nodes are attribute tests and leaves are class labels (True/False).

(i) Decision Tree for A ∧ B (A AND B):

A AND B is TRUE only when BOTH A=True AND B=True.

Decision Tree for A AND B: [A] / \ No Yes | | FALSE [B] / \ No Yes | | FALSE TRUE Truth Table Verification: A=F, B=F -> FALSE (A is No -> immediately FALSE) A=F, B=T -> FALSE (A is No -> immediately FALSE) A=T, B=F -> FALSE (A=Yes, B=No -> FALSE) A=T, B=T -> TRUE (A=Yes, B=Yes -> TRUE)

(ii) Decision Tree for A ∨ [B ∧ C] (A OR (B AND C)):

This is TRUE when A=True, OR when both B=True AND C=True.

Decision Tree for A OR (B AND C): [A] / \ Yes No | | TRUE [B] / \ No Yes | | FALSE [C] / \ No Yes | | FALSE TRUE Truth Table Verification: A=T, B=any, C=any -> TRUE (A=Yes -> immediately TRUE) A=F, B=F, C=any -> FALSE (A=No, B=No -> FALSE) A=F, B=T, C=F -> FALSE (A=No, B=Yes, C=No -> FALSE) A=F, B=T, C=T -> TRUE (A=No, B=Yes, C=Yes -> TRUE)

Answer

Q5. Describe the FIND-S Algorithm [2 marks]

FIND-S Algorithm finds the maximally specific hypothesis consistent with all positive training examples. It starts with the most specific possible hypothesis and generalizes it only when needed to cover positive examples. It completely ignores negative examples.

Algorithm Steps:

- Step 1: Initialize h to the most specific hypothesis: h = <ø,ø,ø,...,ø>
- Step 2: For each POSITIVE training example x:
- For each attribute ai in x:
- If h[ai] = ø (empty): set h[ai] = x[ai] (match this value)
- Else if h[ai] ≠ x[ai]: set h[ai] = ? (generalize to any)
- (Negative examples are IGNORED)
- Step 3: Output final hypothesis h

> EnjoySport Example — Attributes: Sky, Temp, Humidity, Wind, Water, Forecast Init: h = <ø, ø, ø, ø, ø, ø> Ex1 (+): h = Ex2 (+): Compare with h: Humidity differs (Normal vs High) h = [? = any humidity] Ex3 (-): -> IGNORED Ex4 (+): Compare: Water differs (Warm vs Cool), Forecast differs (Same vs Change) h = [generalize those attributes] Final FIND-S hypothesis: Limitation: Only finds ONE hypothesis; ignores negatives; may miss some consistent hypotheses.

PART-B (2 × 10 = 20 Marks) — Long Answer Questions

Long Answer

Q6(a). Summarize the choices in designing a Learning Example with a suitable example [8 marks]

Designing a learning system requires making several fundamental choices. Tom Mitchell's framework identifies the following key decisions that collectively define what the machine learning problem actually is:

Choice 1 — What type of training experience?

- Direct feedback: Output is explicitly given for each input example (e.g. labeled images)
- Indirect feedback: Outcome of a sequence of actions (e.g. win/loss at end of chess game)
- Teacher vs no teacher: Supervised (teacher labels every example) vs Unsupervised (no labels)
- Degree of control: Can the learner choose which examples to learn from? (Active Learning)

Choice 2 — What exactly should be learned (Target Function)?

- Must define precisely what function f: X → Y the system should learn
- For chess: Learn the evaluation function V: Board → numerical value of board position
- For email: Learn f: Email features → {Spam, Not-Spam}
- Can also learn action selection policy, probability distribution, or feature transformation

Choice 3 — What representation for the target function?

- Conjunction of attribute-value pairs: e.g. (concept learning)
- Linear functions: V(b) = w0 + w1*f1 + w2*f2 + ... + wn*fn
- Decision trees: Hierarchical attribute splits
- Neural networks: Weighted interconnections of neurons
- Instance-based: Store examples and retrieve nearest neighbors

Choice 4 — What learning algorithm to use?

- Gradient descent: Minimize error by iteratively adjusting weights
- Version space search: Search hypothesis space (Candidate Elimination)
- Decision tree induction: ID3, C4.5 using information gain
- Backpropagation: Train neural networks using chain rule

Choice 5 — How to evaluate learned function?

- Training accuracy vs test accuracy (generalization)
- k-Fold cross-validation for reliable performance estimate
- Precision, Recall, F1-score for imbalanced datasets

> Complete Example — Designing a Chess-Playing ML System: Training experience: Indirect — game outcome (win/loss) at end Target function: V: BoardState -> numerical evaluation (-inf to +inf) Representation: V(b) = w0 + w1*(num_white_pieces) + w2*(num_black_pieces) + w3*(white_queens) + w4*(mobility) + ... Learning algorithm: Least Mean Squares (LMS) gradient descent V_train(b_t) = V(b_{t+1}) [temporal difference target] wi = wi + n*(V_train(b)-V(b))*fi(b) Evaluation: Win rate against baseline chess engine This design process applies to any ML problem — always ask: "WHAT to learn? FROM WHAT? REPRESENTED HOW? USING WHICH ALGORITHM? EVALUATED HOW?"

Answer

Q6(b). Define the Inductive Learning Hypothesis [2 marks]

The **Inductive Learning Hypothesis** states:

```
"Any hypothesis found to approximate the target function well over a sufficiently
large set of training examples will also approximate the target function well
over other unobserved examples." — Tom Mitchell
```

In simple terms: If a hypothesis h works well on ALL training examples, then we assume it will also work well on future unseen examples drawn from the same distribution.

Key points:

- It is an assumption (inductive leap) — we cannot prove it for a specific problem
- Works best when training data is representative of the real distribution
- Fails when training distribution ≠ test distribution (data shift/bias)
- More training examples = stronger support for the hypothesis

> Example: Training: Learn 100 labeled cat and dog images -> h achieves 98% accuracy Inductive Learning Hypothesis says: h should also achieve ~98% on new cat/dog images we have never seen before. This holds IF: new images are drawn from the same distribution This fails IF: training was only on small dogs, test has large dogs (bias)

Long Answer

Q7(a). EnjoySport Task — Trace the CANDIDATE ELIMINATION Algorithm [5 marks]

Hypothesis space H:

G boundary (most general): **|** S boundary (most specific): <ø,ø,ø,ø,ø,ø>

Training Examples:

Ex	Sky	Airtemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES (+)
2	Sunny	Warm	High	Strong	Warm	Same	YES (+)
3	Rainy	Cold	High	Strong	Warm	Change	NO (-)
4	Sunny	Warm	High	Strong	Cold	Change	YES (+)

> CANDIDATE ELIMINATION ALGORITHM — Step-by-Step Trace: Initialization: S0 = { <ø,ø,ø,ø,ø,ø> } (most specific) G0 = { } (most general) ■■ EXAMPLE 1 (+): -> YES Positive example — must generalize S to cover it: S1 = { } [replace ø with actual values] G: No change (already covers this example) G1 = { } ■■ EXAMPLE 2 (+): -> YES Positive example — S must generalize to cover it: Compare S1 with Ex2: Humidity differs (Normal vs High) -> generalize to ? S2 = { } G: No inconsistency — G2 = { } ■■ EXAMPLE 3 (-): -> NO Negative example — must specialize G to EXCLUDE this example: Current S2 = does NOT cover Ex3 -> S unchanged S3 = { } [no change] Specialize G to exclude : Each specialization must still be consistent with S2: G3 = { , , } (Remove any G member not consistent with S — keep all 3) ■■ EXAMPLE 4 (+): -> YES Positive example — S must cover it: Compare S3= with Ex4: Water differs: Warm vs Cold -> generalize to ? Forecast differs: Same vs Change -> generalize to ? S4 = { } Remove G members inconsistent with Ex4: Check : says Forecast=Same, but Ex4 has Forecast=Change -> REMOVE G4 = { , } ■■ FINAL VERSION SPACE: S = { } (specific boundary) G = { , } (general boundary) Additional question: Is V in H'? H' = {h | S <= h <= G} — check if each hypothesis is between S and G: : Not more general than S (Cold contradicts Warm) -> NOT in H' : More specific than G — check vs S: S= — Sunny matches, ? is ok -> partially consistent This is between G and S -> IN H'

Long Answer

Q7(b). State and Prove Maximum Likelihood Estimation (MLE) Algorithm [5 marks]

Maximum Likelihood Estimation (MLE) finds the hypothesis h that maximizes the probability of observing the training data D given h. It does NOT use prior probabilities.

```
h_MLE = argmax_h P(D | h) = argmax_h PROD_i P(di | h) [assuming i.i.d. examples]
Log-likelihood (easier to compute): h_MLE = argmax_h SUM_i log P(di | h) [log
converts product to sum]
```

MLE for Coin Flipping (Proof):

Let θ = P(Head). Observe data D: n_H heads and n_T tails in n = n_H + n_T flips.

```
Likelihood: L(θ) = P(D|θ) = θ^(n_H) * (1-θ)^(n_T) Log-likelihood: log L(θ) = n_H*log(θ) + n_T*log(1-θ) Maximize by taking derivative and setting to zero: d/dθ [log L] = n_H/θ - n_T/(1-θ) = 0 n_H*(1-θ) = n_T*θ n_H - n_H*θ = n_T*θ n_H = θ*(n_H + n_T) θ_MLE = n_H / (n_H + n_T) [fraction of heads in data]
```

Numerical Example: Observe: 70 Heads, 30 Tails in 100 flips θ_MLE = 70/100 = 0.70 Conclusion: MLE estimates P(Head) = 0.70 Note: MLE can overfit with small samples! 3 Heads, 0 Tails: θ_MLE = 1.0 (always heads) — clearly wrong! Solution: MAP with prior (Bayesian approach) gives more robust estimates

Long Answer

Q8(a). State and Prove Maximum A Posteriori (MAP) Probability Algorithm [5 marks]

MAP (Maximum A Posteriori) finds the hypothesis that maximizes the posterior probability P(h|D) — combining both the likelihood of data AND the prior knowledge about the hypothesis.

```
h_MAP = argmax_h P(h | D) = argmax_h [P(D|h) * P(h)] / P(D) [Bayes Theorem] =
argmax_h P(D|h) * P(h) [P(D) is constant w.r.t h] Log form: h_MAP = argmax_h
[SUM_i log P(di|h) + log P(h)]
```

MAP for Coin Flipping (Proof with Prior):

Prior: P(θ) = Beta(α, β) — Beta distribution (conjugate prior for Binomial)

```
Posterior ∝ Likelihood * Prior: P(θ|D) ∝ θ^(n_H) * (1-θ)^(n_T) * θ^(α-1) *
(1-θ)^(β-1) = θ^(n_H+α-1) * (1-θ)^(n_T+β-1) This is a Beta distribution with
parameters (n_H+α, n_T+β) MAP estimate (mode of posterior): θ_MAP = (n_H + α - 1)
/ (n_H + n_T + α + β - 2) With uniform prior (α=β=1): θ_MAP = θ_MLE =
n_H/(n_H+n_T)
```

Key comparison — MLE vs MAP:

Property	MLE	MAP
No	Yes	
argmax P(D|h)	argmax P(D|h)*P(h)	
Can overfit	More robust	
θ_MLE = θ_MAP	Same result as MLE	
Frequentist	Bayesian	

> Numerical Comparison: Data: 3 Heads, 0 Tails (small sample!) Prior: P(head)=0.5 (fair coin belief), modeled as Beta(2,2) [weak prior] MLE: θ_MLE = 3/3 = 1.0 (claims coin ALWAYS lands heads — extreme!) MAP: θ_MAP = (3+2-1)/(3+0+2+2-2) = 4/5 = 0.8 (more reasonable estimate) With more data (300 H, 0 T): both converge as data overwhelms prior.

Problem

Q8(b). Entropy and Information Gain — Training Examples (A1, A2) [5 marks]

Training examples (6 instances, attributes A1 and A2):

Instance	Classification	A1	A2
1	+	T	T
2	+	T	T
3	-	T	F
4	+	F	F
5	-	F	T
6	-	F	T

SOLUTION Counts: Total = 6, Positive (+) = 3 (instances 1,2,4), Negative (-) = 3 (instances 3,5,6) ■■ PART 1 — ENTROPY of the collection w.r.t target classification: Entropy H(S) = -SUM[p(i) * log2(p(i))] p(+) = 3/6 = 0.5, p(-) = 3/6 = 0.5 H(S) = -(0.5 * log2(0.5)) - (0.5 * log2(0.5)) = -(0.5 * (-1)) - (0.5 * (-1)) = 0.5 + 0.5 = 1.0 bit [Maximum entropy — perfectly mixed] ■■ PART 2 — INFORMATION GAIN of A2 relative to training examples: Split data by A2: A2 = T (instances 1,2,5,6): Instance 1: +, Instance 2: +, Instance 5: -, Instance 6: - Count: 2(+), 2(-), Total = 4 p(+) = 2/4 = 0.5, p(-) = 2/4 = 0.5 H(A2=T) = -(0.5*log2(0.5)) - (0.5*log2(0.5)) = 1.0 bit A2 = F (instances 3,4): Instance 3: -, Instance 4: + Count: 1(+), 1(-), Total = 2 p(+) = 1/2 = 0.5, p(-) = 1/2 = 0.5 H(A2=F) = -(0.5*log2(0.5)) - (0.5*log2(0.5)) = 1.0 bit Information Gain: IG(S, A2) = H(S) - SUM_v [|Sv|/|S| * H(Sv)] = 1.0 - [(4/6)*1.0 + (2/6)*1.0] = 1.0 - [0.667 + 0.333] = 1.0 - 1.0 = 0.0 bits ANSWER: Entropy H(S) = 1.0 bit (maximum — completely mixed classes) IG(S, A2) = 0.0 bits (A2 provides NO information — splits are equally impure!) Conclusion: Attribute A2 is useless for classification in this dataset. A2=T gives 2+,2- (H=1), A2=F gives 1+,1- (H=1) — no improvement over original.

Class Test-II — 2026 | Machine Learning

B.E V-Semester | Department of Computer Science and Engineering | Osmania University

Max Marks: 30 | Part-A: 5×2=10 | Part-B: 2×10=20

PART-A (5 × 2 = 10 Marks) — Short Answer Questions

Answer

Q1. Describe Perceptron and any two activation functions [2 marks]

A **Perceptron** is the fundamental building block of neural networks — a mathematical model of a biological neuron. It computes a weighted sum of inputs, adds a bias, and passes the result through an activation function.

```
Output = f(w1*x1 + w2*x2 + ... + wn*xn + b) = f(W^T*X + b) xi = inputs, wi = learned weights, b = bias, f = activation function
```

Activation Function 1 — Sigmoid:

```
sigma(x) = 1 / (1 + e^(-x)) Range: (0, 1) Derivative: sigma'(x) = sigma(x) * (1 - sigma(x))
```

- Smooth, differentiable — good for output layer in binary classification
- Limitation: Vanishing gradients for very large/small inputs

Activation Function 2 — ReLU (Rectified Linear Unit):

```
ReLU(x) = max(0, x) Range: [0, ∞)
```

- Most popular in hidden layers — computationally efficient
- Solves vanishing gradient problem for positive inputs
- Limitation: Dying ReLU — neurons stuck at 0 for all negative inputs

AND Gate example with Perceptron: Inputs: x1, x2 ∈ {0,1}; Target: x1 AND x2 After training: w1=1, w2=1, b=-1.5 (0,0): 0+0-1.5=-1.5 -> step(-)=0 ✓ (1,1): 1+1-1.5=0.5 -> step(+)=1 ✓ Limitation: Cannot learn XOR (not linearly separable)

Problem

Q2. Coin Problem: P(h1)=0.75, P(h2)=0.25. Determine which coin (fair vs 60% heads) [2 marks]

Two hypotheses: h1 = fair coin (P(H)=0.5), h2 = biased coin (P(H)=0.6). Observation: need to determine from data. Let's assume 5 flips: 3 Heads, 2 Tails (HHTTH) — standard problem.

GIVEN: h1: fair coin P(Head|h1) = 0.5, P(h1) = 0.75 h2: biased coin P(Head|h2) = 0.6, P(h2) = 0.25 Observation D: 5 flips = 3 Heads, 2 Tails Step 1 — Compute Likelihoods P(D|h): P(D|h1) = C(5,3)*(0.5)^3*(0.5)^2 = 10*0.125*0.25 = 10*0.03125 = 0.3125 P(D|h2) = C(5,3)*(0.6)^3*(0.4)^2 = 10*0.216*0.16 = 10*0.03456 = 0.3456 Step 2 — Compute Evidence P(D): P(D) = P(D|h1)*P(h1) + P(D|h2)*P(h2) = 0.3125*0.75 + 0.3456*0.25 = 0.23438 + 0.08640 = 0.32078 Step 3 — Apply Bayes Theorem: P(h1|D) = P(D|h1)*P(h1)/P(D) = 0.23438/0.32078 = 0.7307 ≈ 0.731 P(h2|D) = P(D|h2)*P(h2)/P(D) = 0.08640/0.32078 = 0.2693 ≈ 0.269 CONCLUSION: P(h1|D) = 0.731 > P(h2|D) = 0.269 => MAP decision: The coin is most likely a FAIR COIN (h1) Intuition: Even though biased coin (h2) was slightly more likely to produce 3H,2T data (0.346 > 0.313), the strong prior P(h1)=0.75 dominates, making the fair coin hypothesis more probable overall.

Answer

Q3. Describe: (i) Regularization (ii) K-Fold Cross Validation [2 marks]

(i) Regularization: A technique to prevent overfitting by adding a penalty term to the loss function that discourages large weights.

```
L1 (Lasso): Loss = MSE + lambda * SUM|wi| -> sparse model, feature selection L2 (Ridge): Loss = MSE + lambda * SUM(wi^2)-> shrinks all weights toward zero lambda: controls strength (0=no reg, large=underfitting)
```

(ii) K-Fold Cross Validation: Model evaluation method using all data for both training and testing.

- Split dataset into k equal folds
- For each fold i: train on k-1 folds, test on fold i
- Final score = average of k test scores

5-Fold CV example (100 examples): Run1: Train folds 2-5, Test fold 1 -> 84% Run2: Train folds 1,3-5, Test fold 2 -> 87% ...5 runs total -> Final accuracy = avg(84,87,83,86,85) = 85%

Answer

Q4. Briefly describe the steps to calculate a lower-dimensional subspace of the LDA technique [2 marks]

LDA (Linear Discriminant Analysis) finds projection directions that maximize between-class separation while minimizing within-class variance.

- Step 1: Compute class mean vectors: mu_k = (1/n_k)*SUM_{xi in class k} xi
- Step 2: Compute Within-Class Scatter: Sw = SUM_k SUM_{xi in k} (xi-mu_k)(xi-mu_k)^T
- Step 3: Compute Between-Class Scatter: Sb = SUM_k n_k*(mu_k-mu)(mu_k-mu)^T
- Step 4: Solve generalized eigenvalue problem: Sw^(-1)*Sb * w = lambda * w
- Step 5: Sort eigenvectors by eigenvalue descending
- Step 6: Select top d eigenvectors -> projection matrix W
- Step 7: Project data: Z = W^T * X (d-dimensional space)

```
Optimal direction (2-class): w* = Sw^(-1) * (mu1 - mu2) Max discriminant directions = min(num_features, num_classes - 1)
```

Answer

Q5. What is Reinforcement Learning? What are the elements of Reinforcement Learning? [2 marks]

Reinforcement Learning is a type of machine learning where an agent learns to maximize cumulative reward by interacting with an environment through trial and error. There are no labeled examples — only reward/penalty signals.

> Analogy: Training a dog with treats Agent=dog, Actions=tricks, Reward=treat, Environment=room Real examples: AlphaGo, self-driving cars, robotic arms

Elements of Reinforcement Learning:

- **Agent:** The learner/decision-maker (robot, game player, algorithm)
- **Environment:** Everything the agent interacts with
- **State (S):** Current configuration/situation of environment
- **Action (A):** All possible moves the agent can make
- **Reward (R):** Scalar feedback — positive for good actions, negative for bad
- **Policy (π):** Strategy — maps states to actions (what the agent learns)
- **Value Function V(s):** Expected cumulative reward from state s
- **Q-Function Q(s,a):** Expected reward for taking action a in state s
- **Discount factor γ:** $0 \leq \gamma \leq 1$ — weight of future rewards vs immediate rewards

```
Q-Learning: Q(s,a) <- Q(s,a) + alpha*[r + gamma*max_a' Q(s',a') - Q(s,a)]
```

PART-B (2 × 10 = 20 Marks) — Long Answer Questions

Long Answer

Q6(a). What is a Support Vector? Explain how SVM works with suitable examples [5 marks]

Support Vectors are the data points that lie closest to the decision boundary (hyperplane) — they are the border cases of each class. If any support vector is moved, the decision boundary changes. All other data points do not affect the boundary at all.

How SVM Works — Step by Step:

- Goal: Find the hyperplane that separates two classes with the MAXIMUM margin
- Hyperplane: w^T*x + b = 0 (a line in 2D, plane in 3D, hyperplane in n-D)
- Positive support vectors lie on: w^T*x + b = +1
- Negative support vectors lie on: w^T*x + b = -1
- Margin = distance between the two support vector planes = 2/||w||
- SVM maximizes margin by minimizing ||w||

```
Hard Margin SVM (Linearly Separable Data): Minimize: (1/2)||w||^2 Subject to: yi * (w^T*xi + b) >= 1 for ALL training examples i yi ∈ {+1, -1} — class labels Soft Margin SVM (Non-perfectly Separable): Minimize: (1/2)||w||^2 + C*SUM(xi_i) Subject to: yi*(w^T*xi+b) >= 1-xi_i, xi_i >= 0 C: penalty for violations (Large C=few errors, Small C=large margin)
```

Concrete Example — 2D Classification: Class +1 (spam): Points (2,3), (3,3), (3,4) Class -1 (ham): Points (1,1), (2,1), (1,2) SVM finds: w=[0,1], b=-2.5 (horizontal line at y=2.5) Positive support vectors: closest spam point above line Negative support vectors: closest ham point below line Margin = 2/||w|| = 2/1 = 2 Why SVM is powerful: - Maximum margin = best generalization - Only support vectors matter — robust to outliers far from boundary - Kernel trick handles non-linear boundaries efficiently

---

Long Answer

Q6(b). SVM Kernels: (i) Linear Kernel (ii) Polynomial Kernel (iii) RBF Kernel (iv) Sigmoid Kernel [5 marks]

The **Kernel Trick** maps data to a higher-dimensional space implicitly, allowing SVM to find non-linear boundaries without computing the actual transformation. K(x,z) = ϕ(x)^T·ϕ(z)

(i) Linear Kernel:

```
K(x, z) = x^T * z = x1*z1 + x2*z2 + ... + xn*zn
```

- No transformation — uses original feature space
- Best for linearly separable data with many features
- Fastest computation — O(n) per kernel evaluation
- Use case: Text classification, gene expression data

(ii) Polynomial Kernel:

```
K(x, z) = (x^T*z + c)^d c = constant (bias term, usually 0 or 1) d = degree (d=2 quadratic, d=3 cubic)
```

- Creates polynomial decision boundaries

- d=1 with c=0 → reduces to linear kernel
- Higher d → more complex boundary → risk of overfitting
- Use case: Image recognition, NLP, handwriting recognition

(iii) Radial Basis Function (RBF) / Gaussian Kernel:

```
K(x, z) = exp(-gamma * ||x - z||^2) gamma = 1/(2*sigma^2) — controls width of the Gaussian gamma large -> narrow Gaussian -> complex boundary (overfitting) gamma small -> wide Gaussian -> smooth boundary (underfitting)
```

- Most popular and widely used kernel — works well by default
- K(x,z)=1 when x=z (same point), K→0 as distance increases
- Maps to infinite-dimensional space — highly expressive
- Use case: When no prior knowledge about data distribution

(iv) Sigmoid Kernel:

```
K(x, z) = tanh(alpha * x^T*z + c) alpha = scaling parameter, c = intercept
```

- Mimics a two-layer sigmoid neural network
- Not always a valid kernel (may not be positive semi-definite)
- Use case: When neural-network-like behavior is desired

> Kernel Comparison: Linear: K(x,z) = x^T z -> Fast, simple Polynomial: K(x,z) = (x^T z + 1)^2 -> Curved boundaries RBF: K(x,z) = exp(-||x-z||^2)-> Most flexible (default choice) Sigmoid: K(x,z) = tanh(x^T z) -> Neural-net-like Rule of thumb: Try RBF first; if data is high-dim text use Linear.

Problem

Q7(a). Linear Regression: 10 Patients — Age vs Blood Pressure (Fully Solved) [8 marks]

Patient	1	2	3	4	5	6	7	8	9	10
Age (x)	35	40	38	44	67	64	59	69	25	50
BP (y)	112	128	130	138	158	162	140	175	125	142

LINEAR REGRESSION: y = w1*x + w0 (BP = w1*Age + w0) Formulas: w1 = [n*SUM(xi*yi) - SUM(xi)*SUM(yi)] / [n*SUM(xi^2) - (SUM(xi))^2] w0 = y_bar - w1 * x_bar Step 1 — Raw sums: n = 10 SUM(x) = 35+40+38+44+67+64+59+69+25+50 = 491 SUM(y) = 112+128+130+138+158+162+140+175+125+142 = 1410 x_bar = 491/10 = 49.1 y_bar = 1410/10 = 141.0 Step 2 — Compute SUM(x^2) and SUM(x*y): x^2: 1225+1600+1444+1936+4489+4096+3481+4761+625+2500 = 26157 x*y: (35*112)+(40*128)+(38*130)+(44*138)+(67*158) +(64*162)+(59*140)+(69*175)+(25*125)+(50*142) = 3920+5120+4940+6072+10586+10368+8260+12075+3125+7100 = 71566 Step 3 — Slope w1: w1 = [n*SUM(xy) - SUM(x)*SUM(y)] / [n*SUM(x^2) - (SUM(x))^2] = [10*71566 - 491*1410] / [10*26157 - 491^2] = [715660 - 692310] / [261570 - 241081] = 23350 / 20489 = 1.1396 ≈ 1.14 Step 4 — Intercept w0: w0 = y_bar - w1 * x_bar = 141.0 - 1.14 * 49.1 = 141.0 - 55.974 = 85.026 ≈ 85.03 ANSWER: Regression Line: BP = 1.14 × Age + 85.03 Interpretation: For every 1 year increase in Age, Blood Pressure increases by ~1.14 mmHg A person with Age=0 would have base BP of 85.03 (theoretical intercept) Predictions using the model: Age=35: BP = 1.14*35+85.03 = 39.9+85.03 = 124.93 (actual: 112) Age=44: BP = 1.14*44+85.03 = 50.16+85.03 = 135.19 (actual: 138) Age=67: BP = 1.14*67+85.03 = 76.38+85.03 = 161.41 (actual: 158) Age=69: BP = 1.14*69+85.03 = 78.66+85.03 = 163.69 (actual: 175) Age=25: BP = 1.14*25+85.03 = 28.5+85.03 = 113.53 (actual: 125) Correlation is positive — older patients tend to have higher blood pressure.

Answer

Q7(b). Write short notes on Back Propagation [2 marks]

Backpropagation is the algorithm used to train multilayer neural networks by computing gradients of the loss function with respect to each weight using the chain rule of calculus, then updating weights via gradient descent.

Phase 1 — Forward Pass:

- Feed input X through all layers: a[L] = f(W[L]*a[L-1] + b[L])
- Compute final output y_hat and loss L = (1/2)(y - y_hat)^2

Phase 2 — Backward Pass:

- Output layer delta: d_out = -(y - y_hat) * f'(z_out)
- Hidden layer delta: d_h = (W_out^T * d_out) * f'(z_h)
- Weight gradients: dL/dW = delta * a_prev^T
- Update: W = W - learning_rate * dL/dW

```
Chain Rule: dL/dW1 = (dL/dy_hat)*(dy_hat/dz_out)*(dz_out/da_h)*(da_h/dW1) Each term = local gradient computed at that layer
```

Simple Numerical Example: Input=0.5, Target=1, Weight w=0.6, Learning rate n=0.1 Forward: z=0.6*0.5=0.3, y_hat=sigmoid(0.3)=0.574 Loss = 0.5*(1-0.574)^2 = 0.091 Backward: d = (1-0.574)*0.574*(1-0.574) = 0.426*0.244 = 0.104 Update: w = 0.6 + 0.1*0.104*0.5 = 0.6052 Repeat until loss converges to minimum.

Long Answer

Q8(a). Write notes on: (i) K-Means Algorithm (ii) Multilayer Perceptron [4 marks]

(i) K-Means Algorithm:

K-Means is a partitional, unsupervised clustering algorithm that groups n data points into k clusters by minimizing within-cluster sum of squared distances (WCSS).

```
Objective: Minimize WCSS = SUM_k SUM_{xi in Ck} ||xi - mu_k||^2 mu_k = centroid (mean) of cluster k
```

Algorithm:

- Step 1: Choose k (number of clusters); initialize k centroids randomly
- Step 2: Assign each point to its nearest centroid: c(i) = argmin_k ||xi - mu_k||^2 (Euclidean distance)
- Step 3: Recompute centroids as mean of assigned points: mu_k = (1/|Ck|) * SUM_{xi in Ck} xi
- Step 4: Repeat steps 2-3 until centroids do not change (convergence)

> Example (k=2): Points: A(1,1), B(2,1), C(4,3), D(5,4) Init centroids: mu1=(1,1), mu2=(5,4) Iteration 1 assign: A: d(mu1)=0, d(mu2)=5.0 -> Cluster 1 B: d(mu1)=1, d(mu2)=4.2 -> Cluster 1 C: d(mu1)=3.6, d(mu2)=1.4 -> Cluster 2 D: d(mu1)=5.0, d(mu2)=0 -> Cluster 2 Update: mu1=(1.5,1), mu2=(4.5,3.5) -> Repeat until stable. Limitations: Must specify k, sensitive to initialization, assumes spherical clusters, sensitive to outliers.

(ii) Multilayer Perceptron (MLP):

An MLP is a feedforward neural network with one or more hidden layers. It overcomes the single-layer Perceptron's limitation of only handling linearly separable data.

- Input layer: Receives raw features — one neuron per feature
- Hidden layer(s): Apply non-linear transformations: a = f(W*x + b)
- Output layer: Produces final predictions
- Activation: ReLU in hidden layers, Sigmoid/Softmax in output
- Training: Backpropagation + Gradient Descent

```
Universal Approximation Theorem: An MLP with ONE hidden layer and enough neurons can approximate ANY continuous function to arbitrary precision.
```

> MLP for XOR (which single Perceptron CANNOT learn): XOR: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0 MLP architecture: Input layer: 2 neurons (x1, x2) Hidden layer: 2 neurons with sigmoid activation Output layer: 1 neuron with sigmoid After training: Hidden neuron 1 learns: x1 OR x2 (fires if either is 1) Hidden neuron 2 learns: x1 AND x2 (fires only if both are 1) Output: (x1 OR x2) AND NOT(x1 AND x2) = XOR ✓

Problem

Q8(b). LDA Projection — w1={(4,2),(2,4),(3,6),(4,4)}, w2={(9,10),(6,8),(9,5),(8,7),(10,8)} [6 marks]

LINEAR DISCRIMINANT ANALYSIS — Complete Solution Class w1: (4,2), (2,4), (3,6), (4,4) n1 = 4 Class w2: (9,10), (6,8), (9,5), (8,7), (10,8) n2 = 5

■■ Step 1 — Compute Class Mean Vectors: mu1 = (1/4)*[(4+2+3+4), (2+4+6+4)] = (1/4)*[13, 16] = [3.25, 4.00] mu2 = (1/5)*[(9+6+9+8+10), (10+8+5+7+8)] = (1/5)*[42, 38] = [8.40, 7.60]

■■ Step 2 — Within-Class Scatter Matrix Sw = S1 + S2: For Class w1 (mu1=[3.25, 4.00]): Point Deviation d=(xi-mu1) d*d^T (4,2): d=[0.75,-2.00] [[0.5625,-1.500],[-1.500, 4.000]] (2,4): d=[-1.25, 0.00] [[1.5625, 0.000],[0.000, 0.000]] (3,6): d=[-0.25, 2.00] [[0.0625,-0.500],[-0.500, 4.000]] (4,4): d=[0.75, 0.00] [[0.5625, 0.000],[0.000, 0.000]] S1 = [[2.750,-2.000],[-2.000, 8.000]] For Class w2 (mu2=[8.40, 7.60]): (9,10): d=[0.60, 2.40] [[0.36, 1.44],[1.44, 5.76]] (6, 8): d=[-2.40, 0.40] [[5.76, -0.96],[-0.96, 0.16]] (9, 5): d=[0.60,-2.60] [[0.36, -1.56],[-1.56, 6.76]] (8, 7): d=[-0.40,-0.60] [[0.16, 0.24],[0.24, 0.36]] (10,8): d=[1.60, 0.40] [[2.56, 0.64],[0.64, 0.16]] S2 = [[9.20,-0.200],[-0.200,13.200]] Sw = S1 + S2 = [[11.950,-2.200],[-2.200,21.200]]

■■ Step 3 — Between-Class Scatter Sb: (mu1 - mu2) = [3.25-8.40, 4.00-7.60] = [-5.15, -3.60] Sb = (mu1-mu2)*(mu1-mu2)^T = [[-5.15],[-3.60]] * [[-5.15, -3.60]] = [[(-5.15)^2, (-5.15)*(-3.60)], [(-5.15)*(-3.60), (-3.60)^2]] = [[26.5225, 18.5400], [18.5400, 12.9600]]

■■ Step 4 — Optimal LDA Direction w* = Sw^(-1) * (mu1 - mu2): det(Sw) = 11.95*21.20 - (-2.20)*(-2.20) = 253.34 - 4.84 = 248.50 Sw^(-1) = (1/248.50) * [[21.200, 2.200],[2.200, 11.950]] = [[0.08531, 0.00885],[0.00885, 0.04809]] w* = Sw^(-1) * (mu1-mu2) = [[0.08531, 0.00885],[0.00885, 0.04809]] * [-5.15, -3.60] w*[1] = 0.08531*(-5.15) + 0.00885*(-3.60) = -0.4393 - 0.03186 = -0.4712 w*[2] = 0.00885*(-5.15) + 0.04809*(-3.60) = -0.04558 - 0.17312 = -0.2187 w* = [-0.4712, -0.2187]

■■ ANSWER: Optimal LDA Projection Direction: w* = [-0.471, -0.219] Normalized: ||w*|| = sqrt(0.471^2+0.219^2) = sqrt(0.222+0.048) = sqrt(0.270) = 0.520 w_norm = [-0.906, -0.421] Projected Class Centres (1D): Class w1: w^T*mu1 = -0.471*3.25 + (-0.219)*4.00 = -1.531 - 0.876 = -2.407 Class w2: w^T*mu2 = -0.471*8.40 + (-0.219)*7.60 = -3.956 - 1.664 = -5.620 Separation = |-2.407 - (-5.620)| = 3.213 (well separated in 1D!) Decision boundary (midpoint): (-2.407 + (-5.620))/2 = -4.014 Rule: If w^T*x > -4.014 -> Class w1; If w^T*x < -4.014 -> Class w2

Quick Formula Reference — Both Mid-Terms

All key formulas and definitions at a glance

Useful for last-minute revision before exams

Formula / Concept	Expression
Bayes Theorem	`P(h|D) = P(D|h)*P(h) / P(D)`
MAP estimate	`h_MAP = argmax_h P(D|h)*P(h)`
MLE estimate	`h_MLE = argmax_h P(D|h)`
Entropy	`H(S) = -SUM[p(i)*log2(p(i))]`
Information Gain	`IG(S,A) = H(S) - SUM_v[(|Sv|/|S|)*H(Sv)]`
Naive Bayes	`P(C|x1..xn) ∝ P(C)*PROD P(xi|C)`
Perceptron Output	`y = step(W^T*X + b)`
Perceptron Update	`wi = wi + n*(target-output)*xi`
Sigmoid	`sigma(x)=1/(1+e^-x); sigma'=sigma*(1-sigma)`
ReLU	`ReLU(x) = max(0,x)`
Backprop — output delta	`d_out = (y-y_hat) * f'(z_out)`
Weight update	`W = W - lr * dL/dW`
SVM Objective	`Minimize (1/2)||w||^2 s.t. yi(w^T xi+b)>=1`
SVM Margin	`Margin = 2/||w||`
RBF Kernel	`K(x,z) = exp(-gamma*||x-z||^2)`
Linear Regression w1	`w1 = [n*SUM(xy)-SUM(x)*SUM(y)] / [n*SUM(x^2)-(SUM(x))^2]`
Linear Regression w0	`w0 = y_bar - w1*x_bar`
L1 Regularization	`Loss = MSE + lambda*SUM|wi|`
L2 Regularization	`Loss = MSE + lambda*SUM(wi^2)`
LDA direction	`w* = Sw^(-1)*(mu1-mu2)`
Within-class Scatter	`Sw = SUM_k SUM_{xi in k}(xi-mu_k)(xi-mu_k)^T`
Between-class Scatter	`Sb = SUM_k n_k*(mu_k-mu)(mu_k-mu)^T`
K-Means Objective	`Minimize SUM_k SUM_{xi in Ck}||xi-mu_k||^2`
Q-Learning	`Q(s,a)<-Q(s,a)+alpha*[r+gamma*max Q(s',a')-Q(s,a)]`

FIND-S Update	`if h[ai]=ø: h[ai]=xi[ai]; elif h[ai]≠xi[ai]: h[ai]=?`
Coin MLE	`theta_MLE = n_heads / n_total`